Compiling Ancient Greek Semantic DAGs with Phoneme, Morpheme, and Lexeme Layers

Student Name
New York University

May 2026

Abstract

This paper presents a compiler that turns morphosyntactic analyses from Ancient Greek Universal Dependencies (UD) treebanks into explicit, inspectable, and structurally valid semantic graphs. Each sentence is converted into a directed acyclic graph (DAG) whose nodes preserve surface tokens, lexemes, part of speech, phoneme-layer syllable and segment views, and morpheme-level feature bundles, while edges encode a compact semantic inventory: AGENT, THEME, MODIFIER, COMPLEMENT, and COORD. I evaluate the system through semantic edge accuracy, corpus-scale graph coverage, and node-level phoneme, morpheme, and lexeme diagnostics. First, on a manually labeled historical gold set of 21 real PROIEL and Perseus sentences (236 tokens, 122 gold semantic edges), the complete graph achieves 68.82 precision, 95.90 recall, and 80.14 F1, compared with 19.55 F1 for a local baseline that ignores UD dependency edges and predicts semantic relations only from token order, POS, and case heuristics. Excluding MODIFIER edges raises the score on core semantic roles to 91.75 precision, 94.68 recall, and 93.19 F1, compared with 22.56 F1 for the same baseline. Second, on the full combined Ancient Greek UD test material in the repository (2,353 sentences, 31,913 tokens), the compiler preserves all nodes, produces valid DAGs for every sentence, maps 79.36% of all non-punctuation dependencies, covers 98.98% of content dependencies, and covers 99.96% of predicate-argument dependencies. The main limitation is semantic over-generation of modifier edges, especially for function-like or syntax-only dependents. These results show that UD-backed graph compilation can recover a strong semantic role structure for Ancient Greek while preserving transparent graph structure and full DAG validity.

1 Introduction

Ancient Greek texts are often accessed through translations, commentaries, and lexical notes, but many research questions require explicit structure over the original language. A classic example is determining who did what to whom, under which conditions, and with what modifiers. Dependency treebanks already provide valuable morphosyntactic information for Ancient Greek, but their labels remain primarily syntactic. The central question is whether those syntactic analyses can be compiled into a more semantic, graph-based layer that remains easy to inspect and evaluate.

The system converts UD-annotated sentences into semantic directed acyclic graphs (DAGs). Each token becomes one node. Each node preserves the original token surface form, phoneme-layer syllable and segment output, lemma-level lexeme, part of speech, and UD morphological feature bundle. Syntactic dependency labels are then mapped into a smaller semantic edge inventory: AGENT, THEME, MODIFIER, COMPLEMENT, and COORD. The output is structurally validated with graph checks so that the representation is not only interpretable, but also well formed.

The paper makes four concrete contributions: a compact semantic DAG representation grounded in real treebank inputs, a compiler from UD annotations to that graph inventory, a small manual evaluation set of historical Greek sentences labeled directly in the target format, and an empirical evaluation that combines baseline comparison, corpus-scale coverage statistics, and node-level diagnostics for phoneme, morpheme, and lexeme preservation.

2 Related Work

This work sits at the intersection of dependency annotation, classical treebanking, semantic role labeling, and semantic graph representations.

Universal Dependencies provides a cross-linguistically consistent framework for tokenization, part-of-speech tagging, morphological features, and dependency relations [4]. In this setting, UD matters because it supplies a stable annotation layer from which semantic abstraction can begin. This makes it possible to reorganize syntactic structure into semantic structure while preserving the original token-level linguistic information.

For Ancient Greek specifically, existing dependency treebanks make this kind of work possible. The Ancient Greek and Latin Dependency Treebanks described by Bamman and Crane [1] demonstrate the value of large, manually annotated classical corpora for linguistic and philological research. The PROIEL project [3] extends the treebanking tradition to parallel and historical Indo-European texts, including Ancient Greek material that is already distributed in UD form. These resources provide the morphosyntactic basis for the system input used here.

At the semantic level, the most relevant precedent is semantic role labeling. PropBank-style annotation formalizes predicate-argument structure over events and their participants [5]. The system adopts the same general intuition that subject, object, and clausal dependencies can often be reorganized into semantically meaningful role edges.

The approach is also inspired by graph-based semantic representations such as Abstract Meaning Representation (AMR), which represents sentence meaning as a graph over events, entities, and relations [2]. This paper takes the same graph-oriented perspective while using a compact inventory of node types and role labels tailored to Ancient Greek treebank data.

What distinguishes the present system from the work above is its combination of Ancient Greek UD input, a deliberately small semantic label inventory, explicit graph validation, and evaluation directly on manually labeled historical Greek sentences.

3 Task Definition and Data

3.1 Task Definition

The system input is a sentence with UD-style token-level annotations: surface tokens, lemmas, part-of-speech tags, morphological features, and dependency links. In this paper, *phoneme* refers to the project's operational phoneme layer: a syllable list and character-level segment list produced for each token before morpheme analysis. This is a transparent pipeline layer, not a claim to solve full historical Greek phonology. *Morpheme* refers to the UD morphological feature bundle that encodes inflectional information such as case, number, gender, person, tense, mood, or voice when those features are available. *Lexeme* refers to the lemma-normalized identity of a token. The output is a semantic DAG in which every token is preserved as a node and every retained relation is rewritten into one of five target labels: AGENT, THEME, MODIFIER, COMPLEMENT, or COORD. The task is to compile these token-level annotations into a semantic graph.

The graph is intended to support downstream interpretability. A reader should be able to inspect the output and recover the main event structure, participant roles, modifier attachments, and clausal relationships without consulting the original dependency labels. At the same time, the graph should remain structurally valid: acyclic, internally consistent, and free of malformed edges.

3.2 Treebank Inputs

The repository includes two public Ancient Greek UD test files used in the experiments:

- `data/real_eval/grc_perseus-ud-test.conllu`
- `data/real_eval/grc_proiel-ud-test.conllu`

When converted with the compiler, these files yield 2,353 sentences and 31,913 non-punctuation tokens. These treebanks provide the morphosyntactic analyses from which the semantic DAGs are constructed.

3.3 Manual Historical Semantic Gold

The main semantic evaluation uses `data/gold_semantic/historical_semantic_dags.jsonl`. This file contains 21 manually labeled historical Greek sentences selected from the PROIEL and Perseus UD test materials. The examples total 236 tokens and 122 gold semantic edges in the target inventory.

This manual gold set is intentionally content-semantic. It emphasizes event participants, complements, coordination, and semantically relevant modifiers. Articles, discourse particles, and other purely functional markers are not systematically rewarded. That choice is important because a naive syntax-to-graph mapping could otherwise inflate scores by reproducing formal syntactic detail that does not contribute much to semantic interpretation.

The repository also evaluates a second view of the same manual gold set that removes MODIFIER from scoring. In that setting, the remaining core roles (AGENT, THEME, COMPLEMENT, and COORD) account for 94 gold edges. This core-role subset helps isolate whether the system's main semantic contribution lies in predicate-argument structure or in broad syntax-to-modifier projection.

3.4 Phoneme, Morpheme, and Lexeme Views

The semantic edge evaluation is not the only way to inspect the output. Because Ancient Greek interpretation depends heavily on lexical identity and inflection, the graph also exposes three node-level diagnostic views. The *phoneme view* records each token's syllable split and segment list before downstream morphology. The *morpheme view* checks whether each graph node preserves the available UD morphological feature bundle. The *lexeme view* checks whether each graph node preserves the source lemma.

These diagnostics are deliberately framed as preservation checks rather than as independent phonological reconstruction, lemmatization, or morphological tagging benchmarks. The compiler does not claim to infer historically complete phonemes, new lemmas, or feature bundles from raw text. Instead, it exposes the operational phoneme layer and tests whether graph construction keeps lexical and inflectional evidence aligned with the correct source token. This distinction prevents the evaluation from overstating what the system does while still making clear that lower-level linguistic information remains available for downstream semantic inspection.

4 Methodology

4.1 Layered Raw Pipeline

The project pipeline is explicitly staged:

```
raw text -> phoneme -> morpheme -> lexeme -> semantic token -> relation -> DAG
```

The phoneme stage records each token's syllables and segment sequence. The morpheme stage consumes that prior-layer evidence and emits root, combining-form, or inflectional-ending analyses when the local rules and lexica support them. The lexeme stage then normalizes the token to a lemma-level identity, using the morpheme analysis and available lexical resources. This ordering matters because errors or fallbacks can be traced to the earliest layer where evidence becomes weak, instead of being hidden inside a single opaque graph output.

4.2 Node Construction

The compiler creates one graph node per retained linguistic token. Each node stores the original surface form, phoneme-stage syllables and segments, lemma-level lexeme, UD part of speech, morpheme-level feature bundle, a semantic type, a semantic role, and a confidence score. The semantic typing scheme is intentionally compact. Nouns and proper nouns typically become *entity* nodes, verbs and auxiliaries become *event* nodes, adjectives become *property* nodes, adverbs become *manner* nodes, and conjunction or linker-like items become *connector* nodes. This inventory is designed to support graph inspection, role attachment, and evaluation across real historical sentences.

The compiler copies lemma and morphology information directly into the node representation. This keeps the graph closely aligned with the source treebank annotations and preserves the linguistic details needed for semantic analysis. For the phoneme layer, this means each token remains connected to the syllable and segment output that fed morpheme segmentation. For the lexeme layer, this means that the lemma is retained as an explicit node attribute rather than being hidden inside a token string. For the morpheme layer, this means that inflectional feature bundles are carried through graph construction without being collapsed into the semantic edge label.

4.3 Dependency-to-Semantic Mapping

The core of the system is a rule-based mapping from UD dependency labels into the target semantic edge inventory. Table 1 summarizes the main mapping decisions.

The mapping is intentionally conservative in its label inventory and broad in coverage. Many syntactic distinctions are collapsed into a small number of readable semantic edge types. The trade-off is that some syntactically valid relations, especially modifier-like ones, may become semantically too broad once projected into the smaller label space.

Several UD relations are intentionally not treated as semantic edges. In the corpus-scale coverage analysis, the largest unmapped relation types are `case`, `cc`, `discourse`, `mark`, and `cop`. These are mostly function markers or structural helpers rather than the central content relations the graph is meant to expose. Excluding them makes the graph more semantically readable and explains why content-dependency coverage is much higher than overall dependency coverage.

4.4 Graph Building and Validation

After node and edge creation, the compiler builds a directed graph with stable node identifiers such as `n0`, `n1`, and `n2`. Duplicate edges, self-loops, and malformed links are removed before validation.

Table 1: Main mapping from UD dependency labels to semantic DAG labels.

UD labels	DAG label	Intuition
`nsubj`, `csubj`, `obl:agent`	AGENT	canonical subjects and agentive obliques
`obj`, `iobj`, `nsubj:pass`, `csubj:pass`, `obl:arg`	THEME	objects and patient-like arguments
`xcomp`, `ccomp`, `acl`, `advcl`, `advcl:cmp`, `obl`	COMPLEMENT	clausal and oblique complements
`amod`, `nmod`, `nummod`, `advmod`, `appos`, `det`	MODIFIER	attributive, adverbial, or dependent modifiers
`conj`	COORD	coordination between linked conjuncts

Validation checks confirm that edge labels come from the allowed inventory, endpoints are valid, and the final graph is acyclic. The implementation also verifies predicate-argument well-formedness for required structures and uses `networkx` for cycle detection.

These checks make the output usable as structured data, not just as a readable annotation. High local edge accuracy is not enough if the graph as a whole is invalid.

5 Experiments

5.1 Experiment 1: Manual Historical Gold Evaluation

The first experiment measures exact edge precision, recall, and F1 on the 21 manually labeled historical sentences. A predicted edge is counted as correct only if its source token, destination token, and label all match the gold edge. This metric is strict but appropriate because the output is a graph, not a bag of independent labels. The experiment also reports DAG validity rate.

Two systems are compared on this task. The first is the full compiler described above, which maps UD dependencies directly into semantic graph edges. The second is a deliberately simple baseline. The baseline uses the same UD tokens, lemmas, POS tags, and morphological features, but it ignores UD dependency links. It instead predicts semantic edges from local word order, POS, and case heuristics using the repository's existing linear relation predictor. The comparison asks how much the dependency layer contributes beyond a simple local heuristic.

For both systems, two scores are reported. The *full graph* score evaluates all five labels. The *core roles* score excludes MODIFIER and evaluates only AGENT, THEME, COMPLEMENT, and COORD. This split is motivated by early inspection of the outputs: the compiler appears strong on event structure but less selective about which syntactic dependents deserve a semantic modifier edge.

5.2 Experiment 2: Corpus-Scale UD Conversion

The second experiment measures how the compiler behaves when applied to the full combined Ancient Greek UD test data in the repository. Here the focus is on coverage and robustness over the full converted corpus.

The relevant metrics are:

- overall dependency coverage: the proportion of non-root, non-punctuation UD dependencies that receive a semantic edge;

- content dependency coverage: the same calculation, but excluding relations such as `case`, `cc`, `mark`, `cop`, `discourse`, `aux`, `fixed`, and `flat:name`;
- predicate-argument coverage: the proportion of subject, object, and argument-like dependencies that are converted into semantic edges;
- DAG validity rate: the proportion of converted sentences whose final graphs pass structural validation.

This experiment asks whether the system scales cleanly to thousands of real treebank sentences while preserving the main semantic structure.

5.3 Experiment 3: Phoneme, Morpheme, and Lexeme Diagnostics

The third experiment summarizes the node-level preservation behavior of the compiler. Unlike the semantic edge evaluation, this diagnostic does not compare against a newly annotated gold set. Instead, it audits whether the information already present in the pipeline survives the conversion into graph form.

Four checks are reported. First, *surface-token alignment* verifies that each graph node remains traceable to its source token. Second, *phoneme stage availability* verifies that each token receives syllable and segment output before morpheme segmentation. Third, *morpheme preservation* verifies that each node retains the available UD morphological feature bundle. Fourth, *lexeme preservation* verifies that each node retains the source lemma. These checks are important because later semantic interpretation of Ancient Greek often depends on exactly the information encoded in token form, segmental shape, lemma identity, and inflectional morphology.

5.4 Reproducibility

The main experimental entry points in the repository are `scripts/run_ud_dag.py` for corpus-scale conversion and `scripts/evaluate_historical_gold.py` for manual semantic evaluation. All quantitative results reported below were regenerated locally from those scripts inside the cloned project workspace.

5.5 Project Test Commands

From the current project directory, the following commands run the main local checks and smoke tests used in this repository:

```
cd /Users/eliguli712/DataStructure/ancient_greek_interculator_project

# Install runtime and smoke-test dependencies into the active environment.
python3 -m pip install -U pip
python3 -m pip install -r requirements.txt
python3 - <<'PY'
import nltk
for package in ("punkt", "punkt_tab"):
    nltk.download(package, quiet=True)
print("NLTK tokenizer data ready")
PY
```

Table 2: Comparison between the full compiler and the local baseline on 21 Ancient Greek historical sentences.

System and setting	Precision	Recall	F1	Gold edges
Baseline, full graph	18.06	21.31	19.55	122
Compiler, full graph	68.82	95.90	80.14	122
Baseline, core roles only	21.78	23.40	22.56	94
Compiler, core roles only	91.75	94.68	93.19	94

```
# Unit-test discovery. This repo currently has no pytest-discovered tests,
# so exit code 5 is accepted as a no-op.
PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q || [ "$?" -eq 5 ]

# Gold-set smoke checks.
PYTHONDONTWRITEBYTECODE=1 python3 scripts/evaluate.py \
  --split dev --output outputs/smoke_eval_dev.json
PYTHONDONTWRITEBYTECODE=1 python3 scripts/evaluate.py \
  --split test --output outputs/smoke_eval_test.json
PYTHONDONTWRITEBYTECODE=1 python3 scripts/evaluate_semantic_gold.py \
  --output outputs/smoke_semantic_gold_eval.json
PYTHONDONTWRITEBYTECODE=1 python3 scripts/evaluate_historical_gold.py \
  --output outputs/smoke_historical_semantic_gold_eval.json

# Raw UD-proxy check for lemma, UPOS, semantic type, and relation F1.
PYTHONDONTWRITEBYTECODE=1 python3 scripts/evaluate_ud.py \
  --conllu data/real_eval/grc_perseus-ud-test.conllu \
           data/real_eval/grc_proiel-ud-test.conllu \
  --max-sentences 200 \
  --output outputs/raw_pipeline_ud_eval_200.json

# Phoneme -> morpheme staged output.
PYTHONDONTWRITEBYTECODE=1 python3 src/phoneme.py
PYTHONDONTWRITEBYTECODE=1 python3 scripts/run_phoneme_to_morpheme.py

# End-to-end Aristotle smoke run with staged outputs.
PYTHONDONTWRITEBYTECODE=1 python3 test_aristotle.py \
  --sentences 1 --format staged --output-dir outputs/aristotle_smoke
```

6 Results

Table 2 shows the main comparison between the full compiler and the local baseline on the manual historical gold set.

The comparison is decisive. On the full graph, the compiler improves from 19.55 F1 to 80.14 F1. On core semantic roles, it improves from 22.56 F1 to 93.19 F1. This gap is large enough that it cannot be explained by minor tuning choices. It shows that the UD dependency layer is doing real work for the task, especially for event structure and clausal attachment.

The baseline fails most clearly on complements. Because it ignores dependency links and relies

Table 3: Per-label results for the full compiler on the manual historical gold set.

Label	Precision	Recall	F1	Gold edges
AGENT	80.95	100.00	89.47	17
THEME	93.33	90.32	91.80	31
MODIFIER	38.36	100.00	55.45	28
COMPLEMENT	94.59	94.59	94.59	37
COORD	100.00	100.00	100.00	9

Table 4: Corpus-scale conversion statistics on the combined Ancient Greek UD test files in the repository.

Metric	Value	Notes
Sentences converted	2,353	Perseus + PROIEL test material
Tokens converted	31,913	non-punctuation tokens
Mapped dependencies	23,458	semantic edges produced
Unmapped dependencies	6,102	mostly function markers
Overall dependency coverage	79.36%	all non-root, non-punctuation deps
Content dependency coverage	98.98%	excluding function-marker relations
Predicate-argument coverage	99.96%	subject/object/argument-style deps
DAG validity rate	100.00%	all converted graphs valid

only on local order and morphology, it never recovers the gold COMPLEMENT edges in this evaluation. It also performs weakly on AGENT and THEME, and only modestly on COORD. This is exactly the behavior one would expect from a local heuristic in a language with richer morphology and less rigid word order than English.

Table 3 breaks down the full compiler's per-label results on the same historical evaluation set.

The strongest result is the compiler's performance on the semantic backbone of the sentence. Core roles reach 93.19 F1, with particularly strong performance on COMPLEMENT and perfect performance on COORD in this sample. Both AGENT and THEME also score above 89 F1. These numbers support the claim that the system is capturing predicate-argument structure reliably once strong upstream syntax is available.

The weaker result is the full-graph score of 80.14 F1. The gap between full graph and core roles comes almost entirely from MODIFIER. Modifier recall is 100%, which means the compiler almost never misses a gold semantic modifier once one exists. However, modifier precision is only 38.36%. In other words, the system attaches too many dependents as semantic modifiers. This is consistent with the system design: a broad set of UD syntactic modifiers is collapsed into one semantic label, so purely structural or lightly semantic dependents are often over-produced.

Every evaluated historical sentence still yields a valid DAG. The validity rate is 100%, which means the structural layer of the system is robust even when the semantic edge inventory is imperfect.

Table 4 presents the corpus-scale conversion results on the full combined Ancient Greek UD test material.

These coverage results sharpen the interpretation of the manual gold numbers. The system's lower full-graph precision is not caused by a failure to preserve predicate-argument structure. In fact, predicate-argument coverage is essentially complete at 99.96%. Instead, the remaining gap

Table 5: Phoneme, morpheme, and lexeme diagnostics on the converted Ancient Greek UD test material.

Diagnostic view	What is checked	Result	Interpretation
Surface alignment	Node retains source token identity	100.00%	no token dropout
Phoneme stage	Token has syllable and segment output	100.00%	no prior-layer dropout
Morpheme preservation	Node retains the available UD feature bundle	100.00%	no morphology dropout
Lexeme preservation	Node retains the source lemma	100.00%	no lemma dropout
Independent analysis	Full phonology, lemmatization, or morphology inferred from raw text	not claimed	outside this compiler task

comes from exactly the kinds of dependencies the compiler treats cautiously: function markers and broad modifier-like relations.

The unmapped dependency counts support that interpretation. The largest unmapped categories are `case` (2,322), `cc` (1,660), `discourse` (788), `mark` (580), and `cop` (490). These are mostly structural or functional relations rather than the main semantic relations the system is trying to expose. Once such categories are excluded, content dependency coverage rises to 98.98%.

Table 5 summarizes the node-level phoneme, morpheme, and lexeme diagnostics. The result is intentionally simple: the compiler preserves these fields completely for converted nodes because it treats them as required stage outputs or required node attributes rather than optional annotations. This does not mean that the system has solved historical phonology, lemmatization, or morphological disambiguation from raw text. It means that once the staged pipeline emits phoneme evidence and the UD analysis supplies lemma and morphology, the graph compiler does not discard or misalign that evidence during conversion.

This diagnostic layer clarifies the role of lower-level linguistic evidence in the evaluation. The phoneme stage provides inspectable syllable and segment records for each token; the morpheme and lexeme layers preserve the inflectional and lemma evidence supplied by the treebank. The baseline can use POS and case heuristics, but it lacks the dependency structure needed to place semantic edges reliably. The full compiler keeps the same phoneme, morpheme, and lexeme evidence while also using UD dependencies to recover the semantic backbone of the sentence.

7 Discussion

At its core, the paper addresses a clear technical problem: compiling an existing morphosyntactic analysis into a semantic graph for Ancient Greek. The evaluation shows that this formulation supports strong predicate-argument recovery, transparent graph structure, and reliable large-scale conversion across real treebank data.

The phoneme, morpheme, and lexeme diagnostics make that claim more precise. They show that the compiler is not merely producing unlabeled graph edges; it is preserving the segmental, lemma, and inflectional evidence that a reader needs in order to interpret those edges philologically. The result is therefore a layered representation: phoneme-stage evidence remains visible before morphology, lexeme and morpheme information remain visible on nodes, and semantic roles are made explicit on edges.

The strongest outcome is that a compact semantic label inventory is enough to recover the predicate-argument backbone of real historical Greek sentences. The manual evaluation shows this directly, and the corpus-scale coverage analysis supports it from another angle. The graph

representation is therefore a useful way to expose role structure, clausal structure, and modifier attachment in a form that remains easy to inspect and validate.

The main weakness is modifier over-generation. Several design decisions contribute to this. First, the mapping groups many syntactically different UD relations under one semantic label. Second, the historical gold deliberately omits many function-like or low-content modifiers, so a syntactically faithful projection can look semantically noisy. Third, Ancient Greek often contains particles, determiners, and other dependents whose semantic contribution is subtle or context dependent. A single MODIFIER label is too coarse to handle all of these cases well.

The main limitations extend beyond modifiers. The manual semantic gold set is small, and the system inherits the quality of the upstream UD annotations used as input. The phoneme, morpheme, and lexeme results should therefore be read as preservation diagnostics, not as evidence that the compiler independently solves phonology, lemmatization, or morphological analysis. Even so, the results are strong and consistent. The baseline comparison shows that the final compiler captures much more of the semantic structure than a local heuristic, and the main remaining error source is precise semantic filtering of modifiers rather than event structure itself.

8 Conclusion and Future Work

The paper presents a semantic DAG compiler for Ancient Greek. The system preserves token, phoneme-stage evidence, lexeme, part of speech, morpheme-level morphology, and syntactic structure, then reorganizes those inputs into a compact semantic graph with explicit role labels. On 21 manually labeled historical sentences, the compiler reaches 80.14 F1 on the full graph and 93.19 F1 on core semantic roles, compared with 19.55 and 22.56 F1 for a local baseline that ignores dependency edges. On the full combined UD test files in the repository, it converts 2,353 sentences with 100% DAG validity, near complete coverage of predicate-argument dependencies, and complete node-level preservation of the available phoneme, lexeme, and morpheme attributes.

Future work should focus on the part of the system that the evaluation exposed as fragile: modifier filtering. A separate extension could also evaluate phonological analysis, lemmatization, and morphological tagging against independent gold annotations, but that would be a different task from the graph compiler evaluated here. One direction is to split MODIFIER into a finer taxonomy that distinguishes semantic attributes, determiners, discourse markers, and syntax-only attachments. Another is to learn a pruning or relabeling layer from manually annotated semantic graphs instead of relying on a single deterministic projection. A third is to expand the manual gold set so that evaluation covers more authors, genres, and clause types. Longer-term work could also integrate richer semantic information, such as valency classes, animacy, or limited coreference, while preserving the graph validity guarantees that made the current system attractive.

The result is a reproducible compiler that turns real Ancient Greek treebank analyses into inspectable graph structure.

References

[1] David Bamman and Gregory Crane. The ancient greek and latin dependency treebanks. In *Language Technology for Cultural Heritage*, pages 79–98. Springer, 2011. doi: 10.1007/978-3-642-20227-8_5.

[2] Laura Banarescu, Claire Bonial, Shu Cai, Madalina Georgescu, Kira Griffitt, Ulf Hermjakob, Kevin Knight, Philipp Koehn, Martha Palmer, and Nathan Schneider. Abstract meaning rep-

resentation for sembanking. In *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*, pages 178–186, 2013.

[3] Dag T. T. Haug and Marius L. Jøhndal. Creating a parallel treebank of the old indo-european bible translations. In Caroline Sporleder and Kiril Ribarov, editors, *Proceedings of the Second Workshop on Language Technology for Cultural Heritage Data (LaTeCH 2008)*, pages 27–34, 2008.

[4] Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Yoav Goldberg, Jan Hajič, Christopher D. Manning, Ryan McDonald, Slav Petrov, Sampo Pyysalo, Natalia Silveira, Reut Tsarfaty, and Daniel Zeman. Universal dependencies v1: A multilingual treebank collection. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16)*, pages 1659–1666, 2016.

[5] Martha Palmer, Daniel Gildea, and Paul Kingsbury. The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31(1):71–106, 2005. doi: 10.1162/0891201053630264.